

Proyecto 3 - Tipos y Sistema de Tipos para VeriLang

Juan Camilo Panadero | Sergio Santana

1. Resumen de Cambios

Este documento describe los cambios realizados al proyecto VeriLang del Proyecto 2 para implementar el sistema de tipos requerido en el Proyecto 3. Se partió de la solución propia del Proyecto 2 (20/20). Los cambios cubren los tres criterios de la rúbrica: actualización de la gramática con anotaciones de tipo, verificación de correspondencia de tipos, y regla de existencia de elementos en estructuras.

2. Archivos Modificados y Creados

Archivo	Acción	Descripción
Syntax.rsc	Actualizado	Tipos primitivos y typed literals en Term
AST.rsc	Actualizado	Nuevos constructores para tipos y literales
Checker.rsc	Nuevo	Validación semántica manual sobre el CST
TPChecker.rsc	Nuevo	Integración con TypePal para construcción del TModel
Generator.rsc	Actualizado	Llama al checker y TypePal al final del main()
Plugin.rsc	Actualizado	Summarizer integrado con Checker
Parser.rsc	Nuevo	Entry point para parsear archivos .vl
spec_verilang.vl	Actualizado	Programa válido con tipos anotados
spec_verilang_errors.vl	Nuevo	Programa con errores intencionales para demostrar el checker

3. Cambio 1: Actualización de la Gramática (Criterio 1)

3.1 Tipos primitivos en Type

Se agregaron los tipos primitivos **Int**, **Bool**, **Char** y **String** como alternativas del no-terminal Type. Se usó `right` para hacer `typeArrow` asociativo a la derecha, resolviendo la ambigüedad en tipos como `Set -> Set -> Set`:

```
syntax Type
= typeBase: ID name
| typeInt: 'Int'
| typeBool: 'Bool'
| typeChar: 'Char'
| typeString: 'String'
> right typeArrow: Type left '-\>' Type right;
```

3.2 Typed literals en Term

Se agregaron literales tipados en Term con la sintaxis `valor:Tipo`. Esto permite anotar explícitamente el tipo de un valor concreto en el programa:

```
syntax Term
= termOp: OperatorApp
| termId: ID name
| termParen: '(' Expression ')'
| termInt: INT ':' 'Int' -- ej: 42:Int
| termBool: BoolLit ':' 'Bool' -- ej: true:Bool
| termChar: CHAR ':' 'Char' -- ej: 'a':Char
| termString: STRING ':' 'String' -- ej: "hola":String
| termFloat: FLOAT
| termBare: INT;

syntax BoolLit = bTrue: 'true' | bFalse: 'false';
```

3.3 Palabras reservadas nuevas

Se agregaron los tipos primitivos y literales booleanos al bloque `keyword Reserved` para evitar conflictos con identificadores:

```
keyword Reserved
= ... | "Int" | "Bool" | "Char" | "String" | "true" | "false";
```

4. Cambio 2: Verificación Semántica Manual (Criterio 2 y 3)

Se creó `Checker.rsc` con validación semántica directa sobre el CST usando comprehensions, el mismo mecanismo del Generator. Implementa las siguientes verificaciones:

- **Bases de espacios:** El espacio referenciado en `'defspace A < B end'` debe estar definido como `DefSpace`.
- **Dominios en cuantificadores:** El dominio en `'forall x in D . ...'` y `'exists x in D . ...'` debe ser un espacio definido.
- **Operadores en aplicaciones prefijas:** El operador en `'(op arg1 arg2)'` debe estar definido con `defoperator`.
- **Operadores en aplicaciones infijas:** El operador en `'x op y'` debe estar definido con `defoperator`.

```
// Verifica dominios en forall
for (/(QuantifiedExpr) `forall in . ` := pt)
if ("" notin spaces)
errors += ["Error: space is not defined (used in forall)"];

// Verifica operadores en OperatorApp
for (/(OperatorApp) `( ) ` := pt)
if ("" notin ops)
errors += ["Error: operator is not defined"];
```

5. Cambio 3: Integración con TypePal (Criterio 2 y 3)

Se creó `TPChecker.rsc` que usa `extend analysis::typepal::TypePal` para construir un `TModel` del programa `VeriLang`. `TypePal` registra definiciones y usos de identificadores y verifica su existencia. Se usó `visit` dentro del `collect` del módulo para recorrer el árbol:

```
void collect(current: (Module) `defmodule end`, Collector c) {
  visit(current) {
    case (DefSpace) `defspace end`:
      c.define("", spaceId(), name, defType(userType("")));
    case (DefSpace) `defspace \< end`: {
      c.define("", spaceId(), name, defType(userType("")));
      c.use(base, {spaceId()});
    }
    case (DefOperator) `defoperator : end`: {
      c.define("", operatorId(), name, defType(userType("")));
      collect(tp, c);
    }
    ...
  }
}
```

Los mensajes de tipo `info` de `TypePal` se filtran para mostrar solo errores reales:

```
list[Message] msgs = [m | m <- getMessages(tm), !(m is info)];
```

6. Como Ejecutar y Verificar

6.1 Registrar el lenguaje

Abrir un terminal Rascal nuevo (View -> Command Palette -> Create Rascal Terminal):

```
import Plugin;
main();
```

Esperar que la barra azul inferior termine. Los archivos `.vl` aparecerán con `syntax highlighting`.

6.2 Generar el summary y ejecutar el checker

En el terminal Rascal ejecutar:

```
import Generator;
main();
```

Salida esperada con `spec_verilang.vl` (programa válido):

```
Parsing |project://rascaldsl/instance/spec_verilang.vl|...
```

```

=== VeriLang Module Summary ===
Module: Set
Imports:
using List
Spaces:
SuperSet
Set < SuperSet
Operators:
negation : Bool -> Bool
isIn : Set -> Set -> Bool
implication : Bool -> Bool -> Bool
disjunction : Bool -> Bool -> Bool
union : Set -> Set -> Set [attrs]
Variables:
p : Bool
q : Bool
n : Int
Rules:
(implication p q) -> (disjunction (negation p) q)
Expressions:
(forall x in Set . (forall S in Set . x isIn S))
=====
Written to |project://rascaldsl/instance/output/summary.txt|

=== Semantic Check ===
No semantic errors found.

=== TypePal Check ===
[TypePal] TModel OK - no errors

```

6.3 Verificar detección de errores con Checker

Para demostrar que el checker semántico detecta errores, ejecutar:

```

import Checker;
checkFromFile(|project://rascaldsl/instance/spec_verilang_errors.vl|);

```

Salida esperada:

```

Semantic errors found:
- Error: space NonExistentSpace is not defined
- Error: space UndefinedSpace is not defined (used in forall)

```

El archivo `spec_verilang_errors.vl` contiene intencionalmente:

- `defspace BadSpace < NonExistentSpace end` — base de espacio no definida

- forall y in UndefinedSpace — dominio no definido en cuantificador

6.4 Verificar detección de errores con TypePal

Para ver los errores que TypePal detecta directamente:

```
import TPChecker;
checkTypesFromFile(|project://rascaldsl/instance/spec_verilang.vl|);
```

Salida esperada para programa valido:

```
[TypePal] TModel OK - no errors
```

```
checkTypesFromFile(|project://rascaldsl/instance/spec_verilang_errors.vl|);
```

Salida esperada para programa con errores:

```
[TypePal] Messages:
- error("Undefined space `NonExistentSpace`",
|project://rascaldsl/instance/spec_verilang_errors.vl|(138,16,<9,22>,<9,38>))
```

7. Resumen de Criterios Cubiertos

Criterio	Implementacion	Archivo(s)
Grammar Update & Type Annotations	Tipos Int, Bool, Char, String en Type; typed literals en Term con sintaxis valor:Tipo; BoolLit para true/false	Syntax.rsc, AST.rsc
Type Correspondence Checking	Verifica que operadores y espacios usados en el programa esten definidos. TypePal construye TModel y reporta undefined references.	Checker.rsc, TPChecker.rsc
Data Structure Element Existence Rule	Verifica existencia de bases de espacios (defspace A < B), dominios en forall/exists, y operadores en aplicaciones prefijas e infijas.	Checker.rsc, TPChecker.rsc